ALOUANI Fadi

BTS SIO 1
SLAM
16/10/26

TD 1 : Les TRIGGERS

Sommaire

Introduction.....1
Exercice 1.....1
Exercice 2.....2
Exercice 3.....2
Conclusion.....2

Introduction

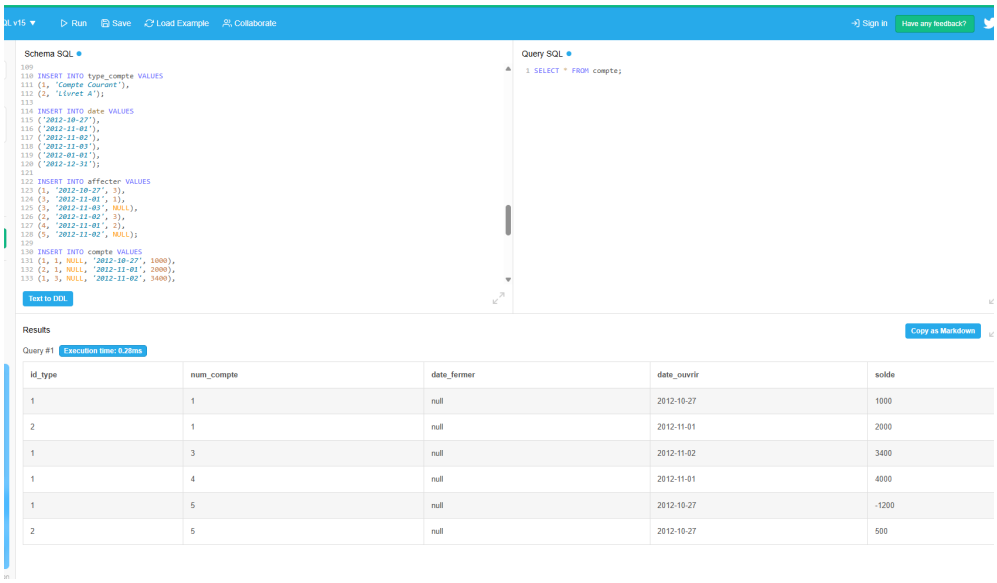
Dans ce TP, j'ai travaillé sur les triggers avec PostgreSQL à partir de la base de données de l'entreprise HHB-Direct. J'ai appris à choisir le type de trigger adapté à une situation et à associer des fonctions à des déclencheurs pour automatiser certaines actions, notamment la mise à jour des soldes des comptes et la vérification des périodes de rémunération.

TP réaliser sur https://www.db-fiddle.com/

ALOUANI Fadi

Exercice 1

Importation de hhb_direct_v1.sql et test.



Fonction virement

J'ai créé une fonction insert_operation_virement() qui permet de retirer le montant du compte source et de l'ajouter au compte destinataire.
J'ai ensuite créé un trigger trig_insert_operation_virement qui se déclenche automatiquement après l'insertion d'une opération de type VIREMENT dans la table operation.

Query SQL

```sql
-- Fonction du trigger
CREATE OR REPLACE FUNCTION insert_operation_virement()
RETURNS TRIGGER AS $$
BEGIN
    UPDATE compte
    SET solde = solde - NEW.montant
    WHERE num_compte = NEW.num_compte
    AND id_type = NEW.id_type;

    UPDATE compte
    SET solde = solde + NEW.montant
    WHERE num_compte = NEW.num_compte_vers
    AND id_type = NEW.id_type_vers;

    RETURN NEW;
END;
$$ LANGUAGE plpgsql;

-- Trigger
CREATE TRIGGER trig_insert_operation_virement
AFTER INSERT ON operation
FOR EACH ROW
WHEN (NEW.type_operation = 'VIREMENT')
EXECUTE FUNCTION insert_operation_virement();
```

ALOUANI Fadi

Test

Results

Copy as Markdown

There are no results to be displayed.

Query #5 Execution time: 0.21ms

id_type	num_compte	date_fermer	date_ouvrir	solde
2	1	null	2012-11-01	2000
1	4	null	2012-11-01	4000
1	5	null	2012-10-27	-1200
2	5	null	2012-10-27	500
1	1	null	2012-10-27	800
1	3	null	2012-11-02	3600

pt2 exercie 1

```
CREATE OR REPLACE FUNCTION virement(
    num_compte_de INTEGER,
    num_compte_vers INTEGER,
    id_type_de INTEGER,
    id_type_vers INTEGER,
    montant INTEGER,
    designation VARCHAR(50)
)
RETURNS INTEGER AS $$
DECLARE
    solde_actuel INTEGER;
BEGIN

    -- Vérifier que le montant est positif
    IF montant <= 0 THEN
        RETURN 0;
    END IF;

    -- Vérifier le compte à débiter
    SELECT solde INTO solde_actuel
    FROM compte
    WHERE num_compte = num_compte_de
    AND id_type = id_type_de
    FOR UPDATE;

    IF NOT FOUND THEN
        RETURN 0;
    END IF;

    -- Vérifier que le solde est suffisant
    IF solde_actuel < montant THEN
        RETURN 0;
    END IF;

    -- Vérifier que le compte destinataire existe
    IF NOT EXISTS (
        SELECT 1 FROM compte
        WHERE num_compte = num_compte_vers
        AND id_type = id_type_vers
    ) THEN
        RETURN 0;
    END IF;

    -- Ajouter la date actuelle
    INSERT INTO date (date)
    VALUES (CURRENT_DATE)
    ON CONFLICT DO NOTHING;

    -- Enregistrer le virement
    INSERT INTO operation (
        id_operation,
        num_compte,
        date,
        id_type,
        id_type_vers,
        num_compte_vers,
        designation,
        type_operation,
        montant
    )
    VALUES (
        (SELECT COALESCE(MAX(id_operation), 0) + 1 FROM operation),
        num_compte_de,
        CURRENT_DATE,
        id_type_de,
        id_type_vers,
        num_compte_vers,
        designation,
        'VIREMENT',
        montant
    );

    RETURN 1;

END;
$$ LANGUAGE plpgsql;

SELECT virement(1, 3, 1, 1, 200, 'Test virement');
```



Query #6 Execution time: 0.27ms

id_type	num_compte	date_fermer	date_ouvrir	solde
1	1	null	2012-10-27	600
1	3	null	2012-11-02	3800

Exercice 2

Pour cet exercice, j'ai créé une fonction appelée insert_nouveau_client() ainsi qu'un trigger nommé trig_insert_nouveau_client. Le trigger se déclenche automatiquement lorsqu'un nouveau client est ajouté dans la table client. J'ai programmé la fonction pour générer automatiquement un identifiant Internet et un mot de passe à partir du pré

-

```sql
-- Fonction du trigger
CREATE OR REPLACE FUNCTION insert_nouveau_client()
RETURNS TRIGGER AS $$
BEGIN
    -- Générer l'identifiant et le mot de passe
    NEW.identifiant_internet :=
        UPPER(LEFT(NEW.prenom_client, 1)) ||
        LOWER(NEW.nom_client);

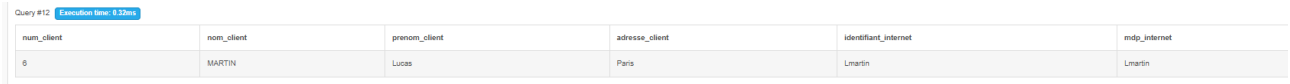
    NEW.mdp_internet := NEW.identifiant_internet;

    RETURN NEW;
END;
$$ LANGUAGE plpgsql;

-- Création du trigger
CREATE TRIGGER trig_insert_nouveau_client
BEFORE INSERT ON client
FOR EACH ROW
EXECUTE FUNCTION insert_nouveau_client();

-- Test
INSERT INTO client (
    num_client, nom_client, prenom_client, adresse_client
)
VALUES (
    6, 'MARTIN', 'Lucas', 'Paris'
);

-- Vérification
SELECT * FROM client WHERE num_client = 6;
```



Query #12 Execution time: 0.32ms

num_client	nom_client	prenom_client	adresse_client	identifiant_internet	mdp_internet
6	MARTIN	Lucas	Paris	Lmartin	Lmartin

Exercice 3

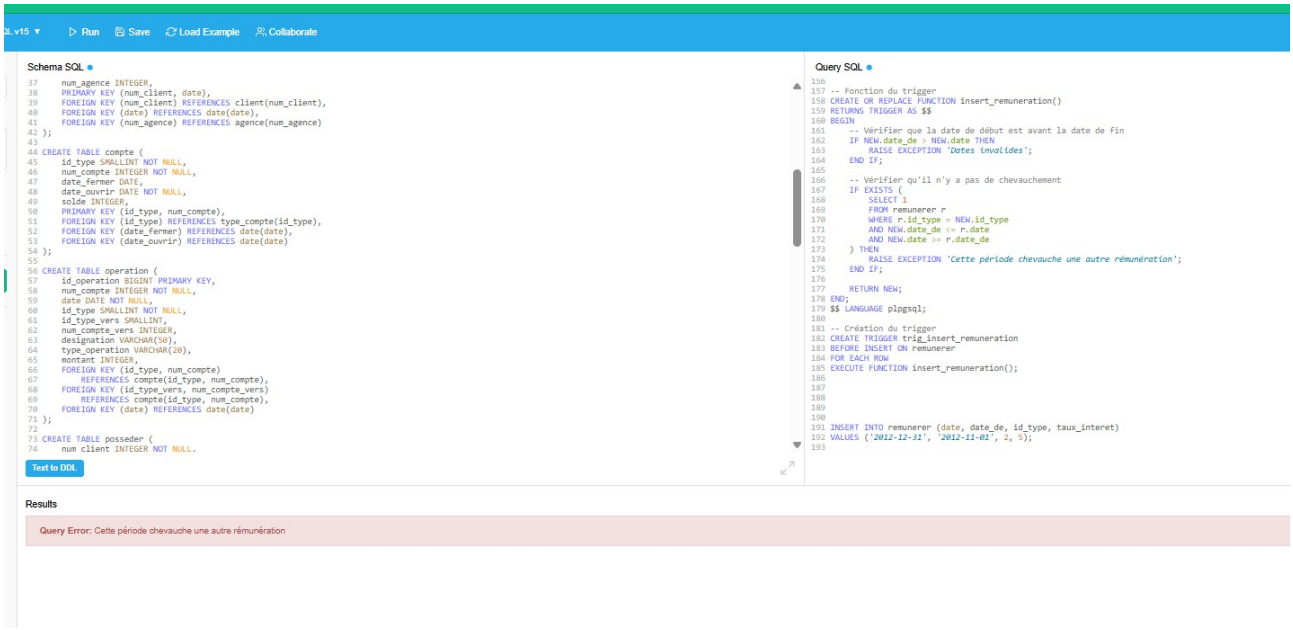
Pour cet exercice, j'ai créé une fonction appelée insert_remuneration() ainsi qu'un trigger nommé trig_insert_remuneration. Ce trigger se déclenche automatiquement lors de l'ajout d'une nouvelle rémunération dans la table remunerer. J'ai programmé la fonction pour vérifier que les dates de la nouvelle période ne chevauchent pas une période déjà enregistrée pour le même type de compte. Enfin, j'ai effectué un test avec une période déjà existante afin de vérifier que l'insertion était bien refusée.

```
-- Fonction du trigger
CREATE OR REPLACE FUNCTION insert_remuneration()
RETURNS TRIGGER AS $$
BEGIN
    -- Vérifier que la date de début est avant la date de fin
    IF NEW.date_de > NEW.date THEN
        RAISE EXCEPTION 'Dates invalides';
    END IF;

    -- Vérifier qu'il n'y a pas de chevauchement
    IF EXISTS (
        SELECT 1
        FROM remunerer r
        WHERE r.id_type = NEW.id_type
        AND NEW.date_de <= r.date
        AND NEW.date >= r.date_de
    ) THEN
        RAISE EXCEPTION 'Cette période chevauche une autre rémunération';
    END IF;

    RETURN NEW;
END;
$$ LANGUAGE plpgsql;

-- Création du trigger
CREATE TRIGGER trig_insert_remuneration
BEFORE INSERT ON remunerer
FOR EACH ROW
EXECUTE FUNCTION insert_remuneration();
```



Conclusion

Ce TP m'a permis de mieux comprendre l'utilisation des triggers avec PostgreSQL et leur lien avec les fonctions SQL. J'ai appris à automatiser certaines opérations et à vérifier les données afin d'éviter les erreurs et de garantir la cohérence de la base de données.